Comparative Study of FLIP, APIC, and PolyPIC

This Option 1 project validates and compares three existing particle-grid transfer schemes for fluid simulation: FLIP, APIC, and PolyPIC. The experiments are run in a shared Taichi framework with fixed grid resolution, particle initialization, boundary treatment, and rendering settings.

The objective is to produce a controlled comparison rather than a new simulator. The main outputs are energy CSV files, videos, visual screenshots, comparison plots, an efficiency benchmark, and this report.

Main finding: PolyPIC has the strongest integrated kinetic-energy retention in the dam-break test. In the liquid-pouring test, FLIP produces much higher kinetic energy, which is treated as possible momentum over-retention or transfer noise rather than an automatic quality improvement. The efficiency benchmark shows the complementary cost tradeoff: FLIP is fastest, APIC is slower, and PolyPIC is slowest.

Course Option, Related Work, and Setup

The course logistics slides describe Option 1 as experimental validation of existing simulators and require the final report to include introduction, related work, methods, results, and discussion. This report follows that structure.

Related work: FLIP is a low-dissipation particle-in-cell variant introduced by Brackbill and Ruppel. APIC improves transfer accuracy by augmenting particles with locally affine velocity information. PolyPIC extends the idea to richer local polynomial functions. Our comparison follows this progression from baseline FLIP to affine and polynomial transfers.

Common setup: 80 x 100 x 80 grid, DX = 0.01, two simulation substeps per rendered frame, 300 output frames, and ratio_970 for the FLIP/PIC blending convention. Two scenes are evaluated: a dense 3D dam break and a liquid pouring configuration.

All CSV outputs are checked for finite kinetic energy values. The APIC and PolyPIC full runs each completed 300 frames for both scenes. The FLIP ratio_970 outputs from the existing main branch are used as the baseline.

Efficiency setup: FLIP, APIC, and PolyPIC were rerun in the same Slurm job on an NVIDIA RTX PRO 6000 Blackwell Server Edition GPU. Rendering and video export were disabled, three repetitions were collected per method-scene pair, and the first five frames were discarded.

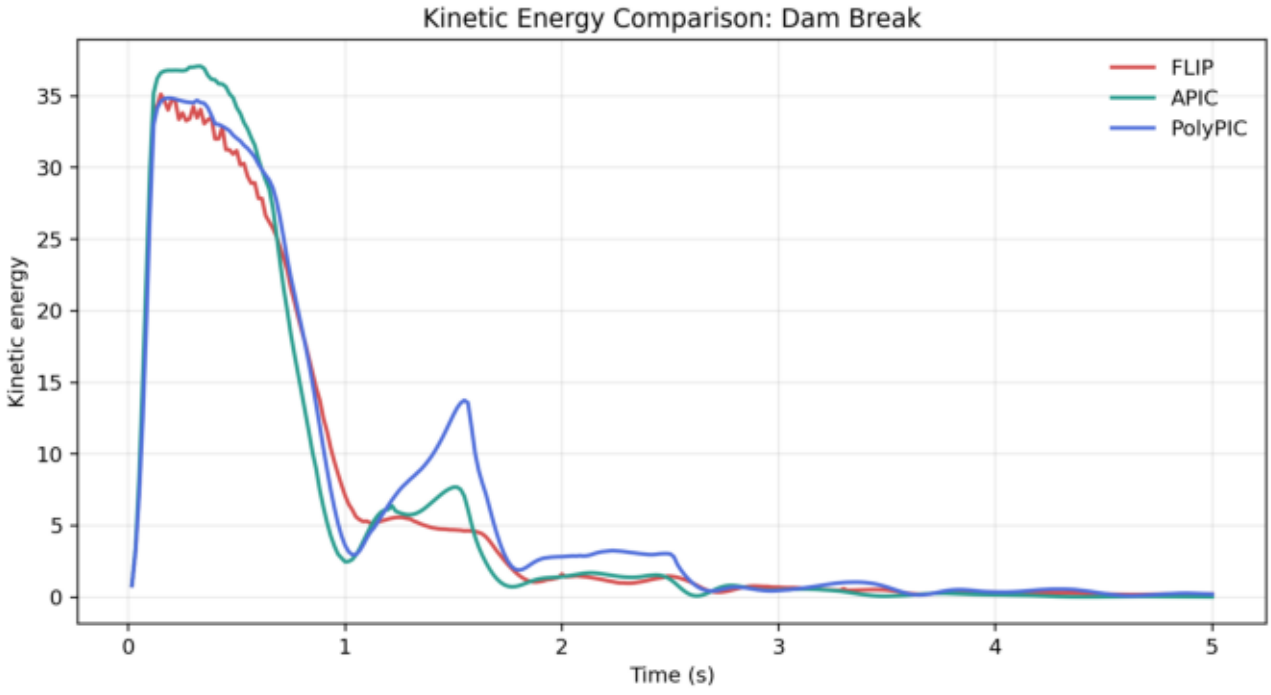
Methods

FLIP transfers particle velocity changes from the grid back to particles, reducing the dissipation of pure PIC but potentially retaining noisy velocity components.

APIC stores a local affine matrix per particle. The P2G transfer evaluates a local velocity model at grid faces, and the G2P transfer reconstructs particle velocity and affine state from the projected grid velocity field. The implementation used here includes finite-value guards and affine damping to keep the full 300-frame runs stable.

PolyPIC generalizes the transfer by carrying higher-order polynomial information. In principle, this can preserve richer local velocity variation than APIC, though it also increases implementation complexity.

Dam Break: Kinetic Energy

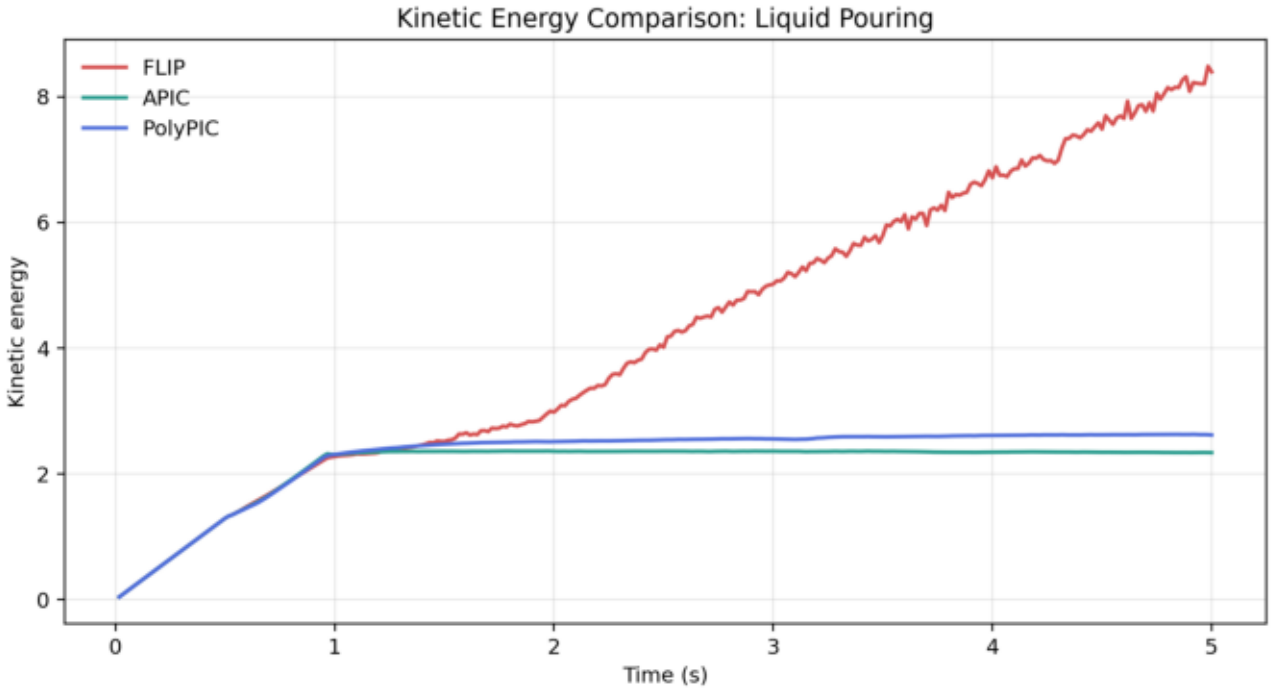


All three methods remain finite for 300 frames. PolyPIC has the largest integrated kinetic energy in this scene, while APIC reaches the highest peak but dissipates more strongly near the end after stabilization.

Summary statistics

Algorithm	Rows	Finite	InitialE	PeakE	FinalE	AUC
FLIP	300	True	0.8193	35.1273	0.2105	30.2746
APIC	300	True	0.8333	37.0836	0.0178	30.2771
PolyPIC	300	True	0.8074	34.8671	0.2117	34.1077

Liquid Pouring: Kinetic Energy

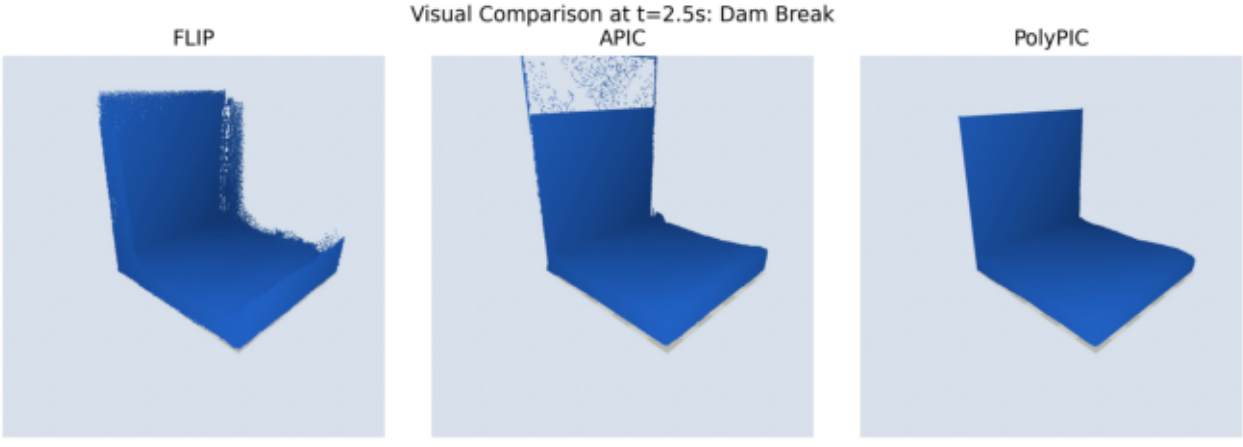


FLIP produces substantially higher kinetic energy in the pouring scene. Because the scene is continuously driven, higher energy is interpreted cautiously: it can indicate reduced dissipation, but also numerical noise or excessive momentum retention.

Summary statistics

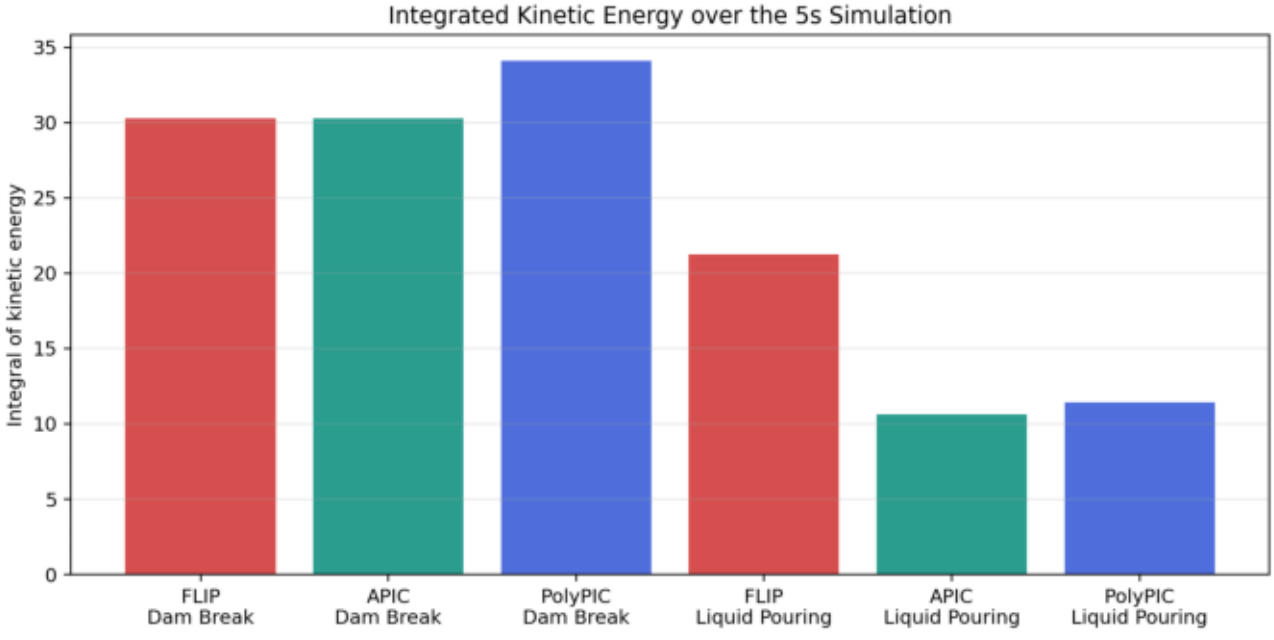
Algorithm	Rows	Finite	InitialE	PeakE	FinalE	AUC
FLIP	300	True	0.0432	8.4851	8.3959	21.2332
APIC	300	True	0.0432	2.3630	2.3384	10.6397
PolyPIC	300	True	0.0432	2.6302	2.6197	11.4223

Visual Comparison



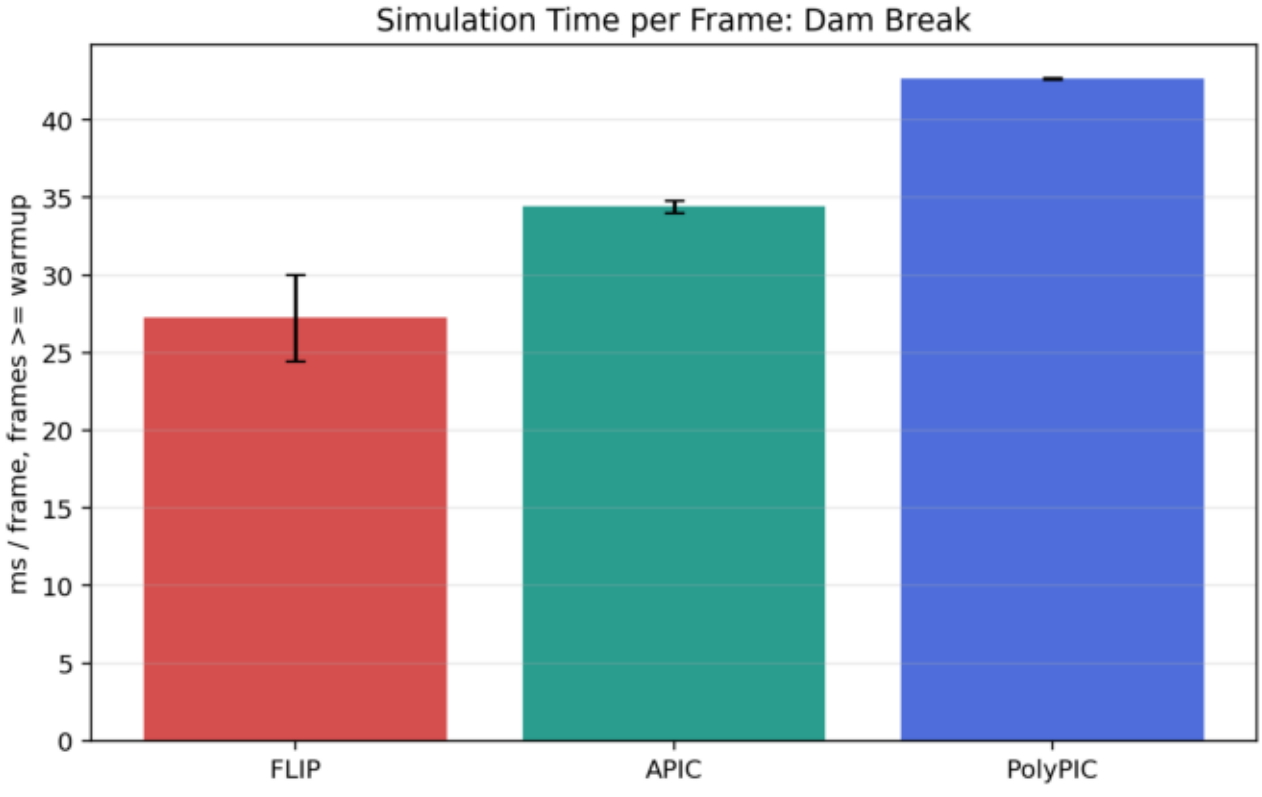
Screenshots extracted at t = 2.5s from the committed dam-break videos. The videos themselves are stored under each algorithm's output directory.

Integrated Energy



Integrated kinetic energy summarizes the total kinetic activity over the five-second run.
This plot is kept separate from the efficiency benchmark so that energy behavior and runtime cost are not mixed.

Efficiency: Dam Break

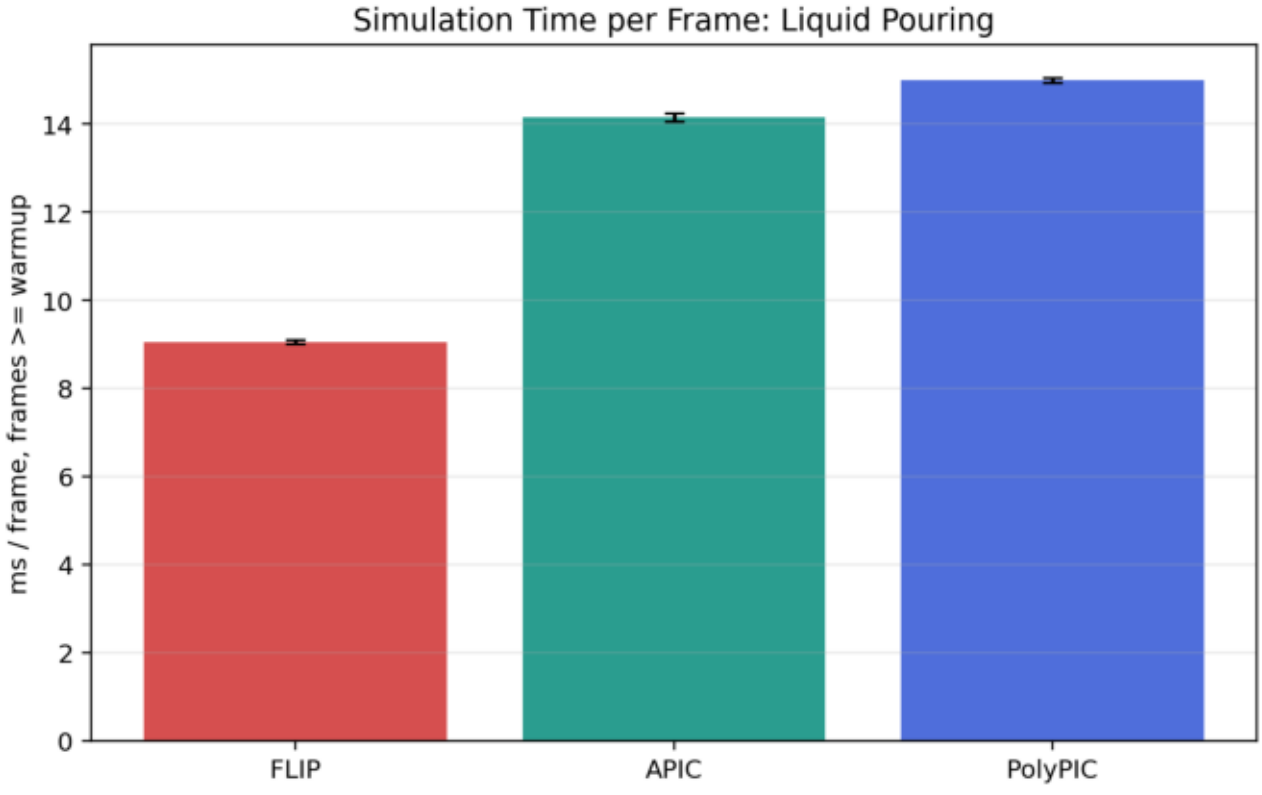


Simulation-only benchmark for the dam-break scene. FLIP is fastest, APIC is moderately slower, and PolyPIC is slowest because it evaluates richer transfer state.

Summary statistics

Algorithm	Mean ms	Std ms	P95 ms	Mpart/s	Speedup
FLIP	27.246	2.777	35.070	75.91	1.00x
APIC	34.417	0.386	45.385	59.67	0.79x
PolyPIC	42.639	0.055	57.272	48.16	0.64x

Efficiency: Liquid Pouring

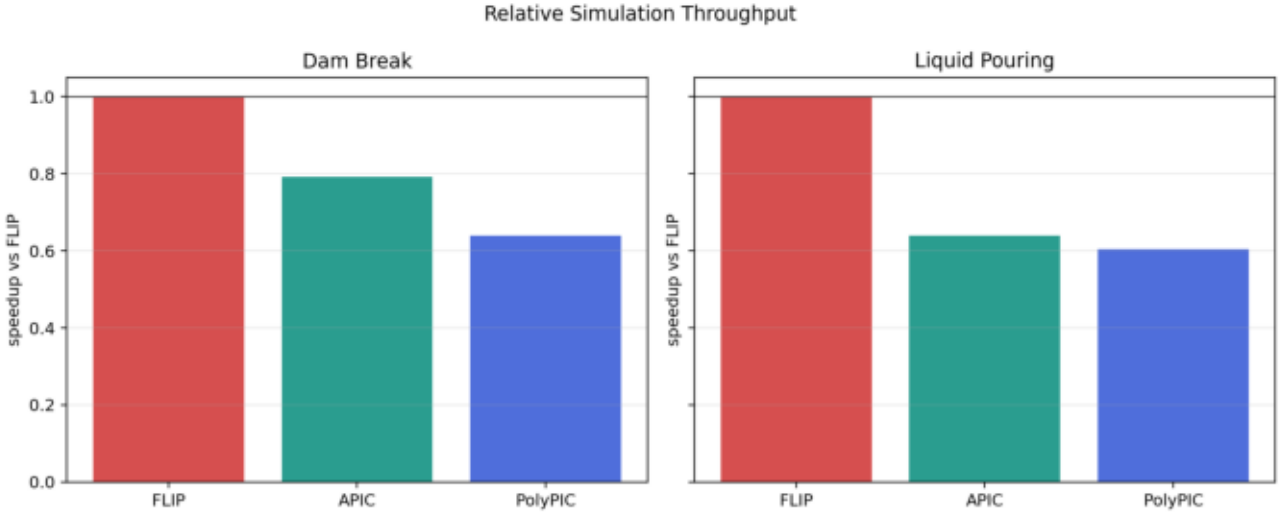


Simulation-only benchmark for the liquid-pouring scene. The same cost ordering appears, with APIC and PolyPIC running at roughly two thirds of FLIP throughput.

Summary statistics

Algorithm	Mean ms	Std ms	P95 ms	Mpart/s	Speedup
FLIP	9.053	0.046	15.317	33.36	1.00x
APIC	14.151	0.092	16.266	21.34	0.64x
PolyPIC	14.988	0.055	16.972	20.15	0.60x

Efficiency: Relative Speedup



Relative speedup is normalized against FLIP within each scene. Values below 1.0 indicate slower runtime than the FLIP baseline.

Discussion, Limitations, and AI Usage

Discussion: The experiments show that a shared framework is essential for meaningful comparison. PolyPIC gives the clearest energy-retention advantage in the dam-break scene. APIC and PolyPIC are smoother than FLIP in liquid pouring, where FLIP's higher kinetic energy should not be read as an unqualified improvement. The efficiency benchmark confirms the expected tradeoff: richer transfer state improves what can be preserved, but reduces throughput.

Limitations: The renderer is particle-based and not a production surface reconstruction. The timing benchmark excludes rendering and video export, so it measures simulation throughput rather than full end-to-end wall time. The APIC implementation uses damping for robustness, so it is not an idealized APIC-only measurement.

AI tool usage: AI assistance was used for code repair, Slurm orchestration, plotting, and report drafting. The numerical values in the tables and plots come from repository CSV files generated by simulation runs, not from language-model invention.

References: Brackbill and Ruppel 1986, doi:10.1016/0021-9991(86)90211-1; Jiang et al. 2015; Fu et al. 2017, doi:10.1145/3130800.3130878; Bridson, Fluid Simulation for Computer Graphics.